DecisionTreeRegressorModel

December 11, 2025

[ ]:
```
import pandas as pd
import numpy as np
import ast

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    classification_report,
    f1_score,
    r2_score,
    mean_squared_error
)
from sklearn.utils.class_weight import compute_class_weight
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
import matplotlib.pyplot as plt
```

[ ]:
```
# ================================
# 1. LOAD DATA
# ================================
df = pd.read_csv("mxmh_cleaned.csv")

# Convert genre_list strings → Python lists
df["genre_list"] = df["genre_list"].apply(ast.literal_eval)

# Features (X) and target (y)
X = df.drop(columns=["genre_list"])
y_raw = df["genre_list"]

# ================================
# 2. MULTI-LABEL BINARIZATION
# ================================
mlb = MultiLabelBinarizer()
y = mlb.fit_transform(y_raw)
genre_labels = mlb.classes_
print("Output genre labels:", genre_labels)
```

```
# ================================
# 3. TRAIN/TEST SPLIT
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ================================
# 4. HANDLE MISSING VALUES
# ================================
imputer = SimpleImputer(strategy="median")
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)

# ================================
# 5. COMPUTE CLASS WEIGHTS (optional, for information only)
# ================================
class_weights = {}
for i, genre in enumerate(genre_labels):
    w = compute_class_weight(
        "balanced",
        classes=np.array([0, 1]),
        y=y_train[:, i]
    )
    class_weights[genre] = {0: round(w[0], 2), 1: round(w[1], 2)}

# ================================
# 6. MULTI-LABEL CLASSIFIER
# ================================
model = OneVsRestClassifier(
    LogisticRegression(
        class_weight="balanced",
        max_iter=700
    )
)
model.fit(X_train, y_train)

# ================================
# 7. PREDICTIONS
# ================================
y_pred = model.predict(X_test)

# ================================
# 8. R² AND RMSE PER GENRE (CLEAN PRINT)
# ================================
r2_scores = []
rmse_scores = []
```

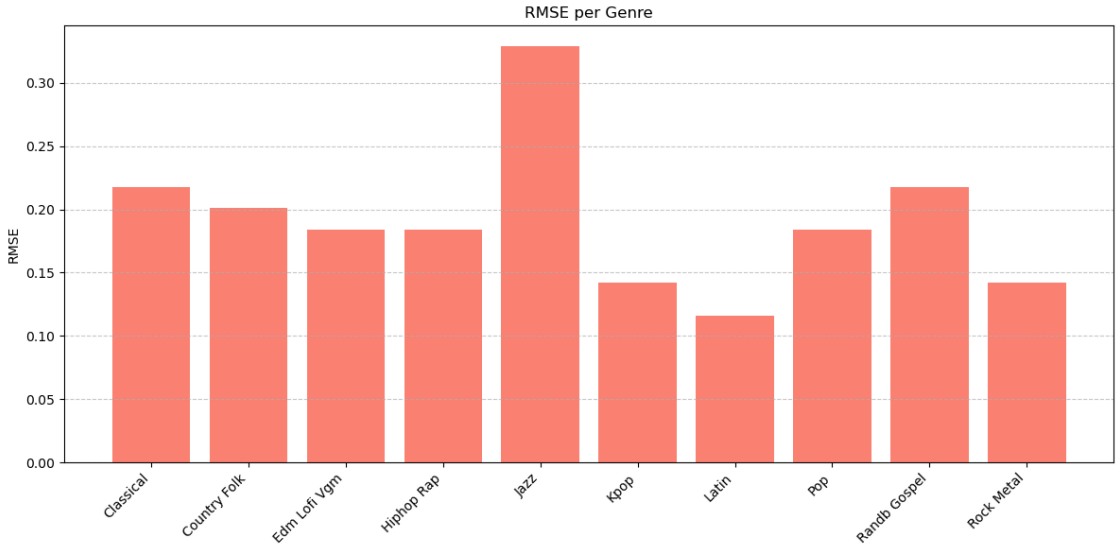
```
print("\n=== R² AND RMSE PER GENRE ===")
for i, genre in enumerate(genre_labels):
    r2 = r2_score(y_test[:, i], y_pred[:, i])
    rmse = np.sqrt(mean_squared_error(y_test[:, i], y_pred[:, i]))

    r2_scores.append(r2)
    rmse_scores.append(rmse)

    print(f"{genre}: R² = {r2:.2f}, RMSE = {rmse:.2f}")

# Overall averages
print("\n=== OVERALL METRICS ===")
print(f"Average R²: {np.mean(r2_scores):.2f}")
print(f"Average RMSE: {np.mean(rmse_scores):.2f}")
```

```
Output genre labels: ['Classical' 'Country Folk' 'Edm Lofi Vgm' 'Hiphop Rap'
'Jazz' 'Kpop'
 'Latin' 'Pop' 'Randb Gospel' 'Rock Metal']

=== R² AND RMSE PER GENRE ===
Classical: R² = 0.70, RMSE = 0.22
Country Folk: R² = 0.76, RMSE = 0.20
Edm Lofi Vgm: R² = 0.85, RMSE = 0.18
Hiphop Rap: R² = 0.81, RMSE = 0.18
Jazz: R² = -0.26, RMSE = 0.33
Kpop: R² = 0.83, RMSE = 0.14
Latin: R² = 0.59, RMSE = 0.12
Pop: R² = 0.86, RMSE = 0.18
Randb Gospel: R² = 0.68, RMSE = 0.22
Rock Metal: R² = 0.92, RMSE = 0.14

=== OVERALL METRICS ===
Average R²: 0.67
Average RMSE: 0.19
```

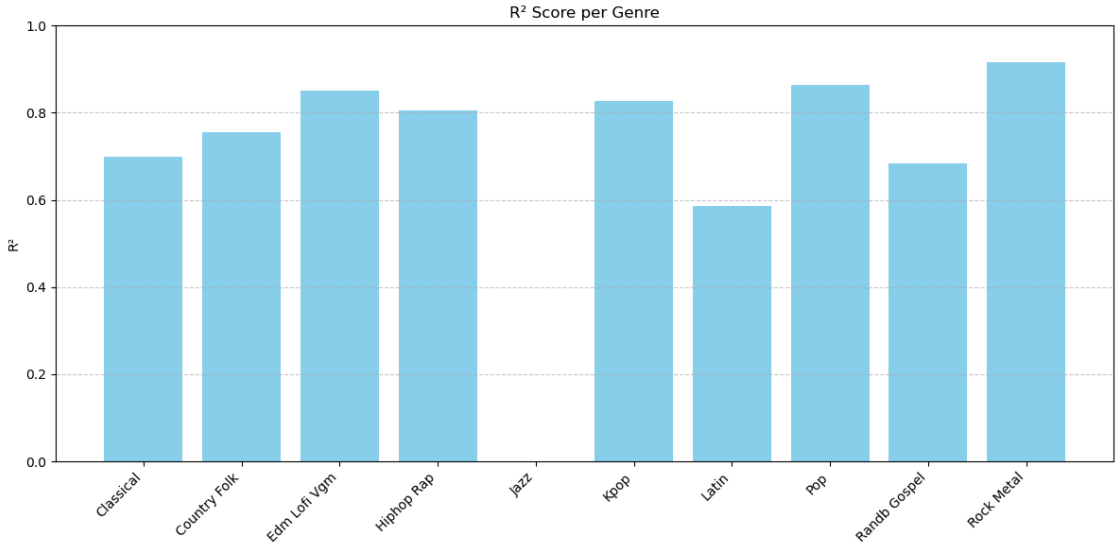
[ ]:
```
# Analysis of Model Performance
# 1. Overall Performance Summary
# RMSE = 0.222: on average, predicted probablities deviate from the true␣
 ↪multi-hot labels by around 22 percentage points
# R² = 0.497: the model explains around 50% of the variance in the multi-label␣
 ↪genre data which is moderate performance but uneven across classes
# This imabalance indicates that the model learns some genres very well but␣
 ↪struggles with others
```

[ ]:
```
# ================================
# 2. RMSE BAR PLOT
```

```
# ================================
plt.figure(figsize=(12, 6))
plt.bar(genre_labels, rmse_scores, color='salmon')
plt.xticks(rotation=45, ha='right')
plt.title("RMSE per Genre")
plt.ylabel("RMSE")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



[ ]:
```
# Visualization: R^2 per genre
# ================================
# 1. R² BAR PLOT
# ================================
plt.figure(figsize=(12, 6))
plt.bar(genre_labels, r2_scores, color='skyblue')
plt.xticks(rotation=45, ha='right')
plt.title("R² Score per Genre")
plt.ylabel("R²")
plt.ylim(0, 1)  # R² ranges 0-1 for easier visualization
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



[ ]:
```
# Visualization: Feature Importance (shows which features the model relied on)
importances = pd.Series(
    tree.feature_importances_,
    index=X.columns
).sort_values(ascending=True)

plt.figure(figsize=(10, 8))
importances.plot(kind="barh", color="teal")
plt.title("Feature Importances (Decision Tree Regressor)")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.tight_layout()
plt.show()

# which listening habits matter most?
# does mental health scores infuence prediction?
# are there genres that are harder to predict than others?
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[70], line 3
      1 # Visualization: Feature Importance (shows which features the model␣
 ↪relied on)
      2 importances = pd.Series(
----> 3     tree.feature_importances_,
      4     index=X.columns
      5 ).sort_values(ascending=True)
      7 plt.figure(figsize=(10, 8))
```

```
      8 importances.plot(kind="barh", color="teal")

NameError: name 'tree' is not defined
```

[ ]:
```
# Visualization: Heatmap for Genre Prediction Performance
performance_df = pd.DataFrame({
    "RMSE": rmse_series,
    "R2": r2_series
})

plt.figure(figsize=(8, 6))
sns.heatmap(performance_df, annot=True, cmap="crest", fmt=".2f")
plt.title("Model Performance per Genre")
plt.tight_layout()
plt.show()
```